Descripción y formulación del proyecto

Proyecto: ProcSim Local · **Asignatura:** Sistemas Operativos · **Grupo:** 10 **Programa:** Ingeniería de Software · **Universidad:** Manuela Beltrán · **Fecha:** noviembre de 2026

---

1. Nombre del proyecto

ProcSim Local: simulador de escritorio de procesos, hilos y concurrencia para Windows.

El nombre une *Proc* (procesos) y *Sim* (simulador). *Local* indica que funciona en el equipo del usuario, sin internet ni servidor.

2. Descripción general del producto

ProcSim Local es una aplicación de escritorio para Windows que muestra de forma visual e interactiva lo que hace un sistema operativo cuando crea y administra procesos e hilos. También muestra cómo aparecen y se controlan los problemas de concurrencia.

El usuario elige un escenario (por ejemplo, "condición de carrera"), configura sus parámetros (número de hilos, prioridad, mecanismo de sincronización) y la aplicación lo ejecuta paso a paso. En pantalla se ven el diagrama de estados, el bloque de control de cada proceso (PCB), la línea de tiempo de los hilos y una bitácora de eventos. Cada ejecución se guarda en una base de datos local (SQLite) para revisarla después o exportarla a `.txt`.

Los procesos son **simulados**: el programa no crea ni controla procesos reales del equipo. En cambio, los escenarios de concurrencia usan **hilos reales** de Python (`threading`), que el sistema operativo Windows planifica como hilos nativos. Así, la condición de carrera y el interbloqueo que se ven en pantalla ocurren de verdad.

3. Planteamiento de la necesidad o problemática

3.1 Problemática

En la asignatura de Sistemas Operativos, los temas de procesos, hilos y concurrencia se estudian sobre todo en teoría: diapositivas, diagramas estáticos y ejercicios en papel. Esto genera varias dificultades:

1. **Conceptos abstractos y difíciles de visualizar.** Los estados de un proceso, el PCB o el cambio entre modo usuario y modo kernel ocurren dentro del sistema operativo y el estudiante no puede verlos.
2. **Problemas de concurrencia que no se ven a simple vista.** La condición de carrera, el interbloqueo y la inanición dependen del orden en que se ejecutan los hilos. Es difícil entender por qué ocurren y cómo se corrigen sin verlos en acción, y cuando aparecen en un programa real no siempre se repiten.
3. **Poca práctica.** Las herramientas reales para observar procesos (administradores de tareas, depuradores o comandos de consola) son complejas, muestran mucha información a la vez y no

están pensadas para enseñar. Otras opciones requieren Linux, máquinas virtuales o instalaciones especiales.

4. **Dificultad del docente para demostrar en clase.** No hay una herramienta sencilla con la que el docente proyecte un ejemplo paso a paso, lo pause, lo repita y compare el resultado con y sin protección.

3.2 Necesidad

Se necesita una **herramienta sencilla, local y visual** que permita a estudiantes y docentes:

- Ver cómo un sistema operativo crea, administra y termina procesos e hilos.
- Reproducir los problemas de concurrencia y comparar el resultado sin protección contra el resultado con mutex o semáforo.
- Controlar la simulación (iniciar, pausar, avanzar paso a paso, cambiar la velocidad y reiniciar) para analizar cada evento.
- Funcionar en Windows sin internet, sin servidor y sin instalaciones complejas.
- Guardar el historial de las simulaciones para revisarlas y exportarlas.

4. Justificación de la solución

- **Aprender viendo y haciendo.** Un simulador convierte conceptos abstractos en algo observable. El estudiante cambia un parámetro y ve el efecto de inmediato, lo que refuerza lo visto en clase.
- **Problemas reales, de forma controlada.** Los escenarios de concurrencia usan hilos reales del sistema operativo. El error de la condición de carrera es real, pero el simulador lo hace repetible y lo explica.
- **Bajo costo y fácil acceso.** Usa Python y bibliotecas estándar (Tkinter, `threading`, `sqlite3`), sin licencias pagas. Se puede distribuir como un ejecutable `.exe` que funciona sin instalar nada más.
- **Pensado para Windows**, que es el sistema operativo de la mayoría de equipos de estudiantes y salas de cómputo, sin depender de máquinas virtuales.
- **Apoyo al docente.** Permite proyectar ejemplos en clase, repetirlos y exportar la bitácora como evidencia.
- **Coherencia con la asignatura.** Cada función corresponde a un tema del programa de Sistemas Operativos (ver la sección 10).

5. Objetivo general

Desarrollar una aplicación de escritorio para Windows que simule, de forma visual e interactiva, cómo un sistema operativo **crea y administra procesos e hilos**, y cómo aparecen y se controlan los problemas de concurrencia: **condición de carrera, interbloqueo e inanición**.

6. Objetivos específicos

1. Implementar un módulo de **gestión de procesos** que permita crear y finalizar procesos, consultar su PCB y animar su ciclo de vida.
2. Implementar un módulo de **hilos y modos de ejecución** que compare hilos con procesos independientes y muestre el paso de modo usuario a modo kernel en una llamada al sistema y en una interrupción.
3. Implementar un módulo de **concurrencia** con hilos reales que reproduzca la condición de carrera, la sección crítica con mutex y semáforo, el interbloqueo y la inanición.
4. Diseñar una **interfaz gráfica** con controles de simulación (iniciar, pausar, paso a paso, velocidad y reiniciar) y una explicación teórica en cada escenario.
5. Registrar cada evento en una **base de datos local SQLite** y permitir exportar la bitácora a `.txt`.
6. Validar el funcionamiento con **casos de prueba** (`unittest`) y documentar el proyecto en GitHub (README y manual de usuario).

7. Alcance inicial del producto

Incluye

- Prototipo funcional de escritorio, ejecutable en Windows 10/11.
- Las 9 funciones descritas en la sección 9.
- Hilos reales de Python en los escenarios de concurrencia.
- Base de datos local (SQLite) para guardar escenarios, ejecuciones y bitácora.
- Repositorio público en GitHub, README y manual de usuario.
- Casos de prueba con capturas y análisis de resultados.

No incluye

- Crear o modificar un kernel real: los procesos son simulados.
- Gestión de memoria, sistema de archivos ni planificación avanzada de CPU.
- Versiones web, móvil, Linux o macOS.
- Inicio de sesión, base de datos en servidor o modo multiusuario.
- Control de los procesos reales del equipo del usuario.

Alcance de esta primera entrega

Planeación y estructura técnica: documentación, diagramas, modelo de datos, estructura de carpetas y repositorio. La implementación del código se hará en las siguientes fases (ver el README, sección "Estado actual y próximos avances").

8. Usuarios o beneficiarios

Usuario	Descripción	Cómo usa el producto
Estudiante	Estudiante de Sistemas Operativos o de asignaturas afines.	Ejecuta los escenarios, cambia parámetros y observa los resultados para entender los conceptos.
Docente	Profesor de la asignatura.	Proyecta ejemplos en clase, los pausa y los repite paso a paso; exporta la bitácora.
Beneficiario indirecto	El programa académico.	Cuenta con un recurso de apoyo libre y reutilizable para la asignatura.

9. Funcionalidades principales

El simulador ofrece **9 funciones** organizadas en 3 módulos.

Módulo A · Gestión de procesos

#	Función	Descripción	Resultado en pantalla
F1	Crear y finalizar procesos	Crear procesos con nombre, prioridad y tiempo de ráfaga; listarlos y terminarlos.	Lista de procesos activos
F2	Consultar el PCB	Ver el bloque de control de cada proceso: PID, estado, contador de programa, prioridad y registros.	Ficha PCB actualizada en vivo
F3	Ciclo de vida	Animar las transiciones entre nuevo, listo, ejecución, bloqueado y terminado.	Diagrama de estados animado

Módulo B · Hilos y modos de ejecución

#	Función	Descripción	Resultado en pantalla
F4	Hilos de un proceso	Crear varios hilos dentro de un proceso y compararlos con procesos independientes: memoria compartida frente a pila y registros propios.	Línea de tiempo por hilo
F5	Modo usuario ↔ modo kernel	Simular una llamada al sistema (por ejemplo, `read()`) y una interrupción de E/S, mostrando el cambio de modo y el regreso.	Secuencia de pasos explicada

Módulo C · Concurrencia

#	Función	Descripción	Resultado en pantalla
F6	Condición de carrera	Dos hilos incrementan un contador compartido sin protección.	Valor esperado vs. valor real
F7	Sección crítica	Repetir el caso anterior usando mutex o semáforo.	Resultado correcto y comparación

#	Función	Descripción	Resultado en pantalla
F8	Interbloqueo	Dos hilos retienen un recurso cada uno y esperan el del otro.	Grafo de espera circular
F9	Inanición	Un hilo de baja prioridad nunca obtiene el recurso.	Tiempo de espera por hilo

En todos los escenarios

- **Controles:** iniciar, pausar, paso a paso, velocidad y reiniciar.
- **Bitácora** de eventos, guardada en la base de datos y exportable a `.txt`.
- **Explicación** breve del concepto teórico en pantalla.

10. Relación del proyecto con los conceptos de Sistemas Operativos

Concepto de Sistemas Operativos	Dónde aparece en ProcSim Local
Proceso y su ciclo de vida (nuevo, listo, ejecución, bloqueado, terminado)	F1 y F3: creación, finalización y diagrama de estados animado.
Bloque de control de proceso (PCB)	F2: ficha con PID, estado, contador de programa, prioridad y registros.
Planificación y cambio de contexto	F3: despacho de listo a ejecución e interrupción de ejecución a listo, según la prioridad y la ráfaga.
Hilos (memoria compartida, pila y registros propios)	F4: hilos de un mismo proceso frente a procesos independientes.
Modo usuario y modo kernel, **llamadas al sistema** e **interrupciones**	F5: secuencia de una llamada `read()` y de una interrupción de E/S.
Condición de carrera	F6: dos hilos reales modifican un contador compartido sin protección.
Sección crítica y exclusión mutua (mutex y semáforo)	F7: `threading.Lock` y `threading.Semaphore` corrigen el resultado.
Interbloqueo (condiciones de Coffman, grafo de asignación de recursos)	F8: espera circular entre dos hilos y dos recursos, mostrada como grafo.
Inanición y prioridades	F9: un hilo de baja prioridad que no obtiene el recurso; tiempo de espera por hilo.
Hilos del sistema operativo	La aplicación usa `threading`, que en Windows crea hilos nativos planificados por el propio sistema operativo.